CSF2753: Web Service Development — Coursework 1

RESTful Web Service Design Document

Student number: 2356784 **Service root URI:** `http://localhost:8181/2356784/` **Dataset theme:** DVLA Vehicle Service Transactions

---

Introduction

This document describes the design of a RESTful web service that provides access to, and operations on, a dataset of DVLA-style vehicle service transactions. Each transaction records a single interaction a customer has with the agency — for example taxing a vehicle, declaring a vehicle off-road (SORN), registering a change of keeper, or renewing a driving licence — together with the channel the customer used, the amount paid, and the current processing status.

The design follows the six REST constraints introduced in the module (uniform interface, stateless, cacheable, client–server, layered system, code-on-demand). In particular, resources are modelled as nouns identified by URIs, representations are exchanged in JSON, and every request carries all the information the server needs to process it (statelessness). The design targets a future Spring Boot implementation but does not depend on any particular framework.

Task 1: Dataset Selection and Description

1.1 Choosing a Dataset

The chosen dataset is a **vehicle service transactions ledger**: a flat, tabular record of individual transactions processed by a vehicle licensing agency across multiple service channels (web, telephone, agent, and post office). The dataset is synthetic but is modelled directly on the structure of real public transactional data — for example the UK Government's digital-service usage and transactions statistics (https://www.gov.uk/search/research-and-statistics) and publicly available e-commerce transaction datasets such as *Online Retail II* on the UCI Machine Learning Repository (https://archive.ics.uci.edu/datasets) and Kaggle (https://www.kaggle.com/datasets), all of which share the same shape: a unique identifier, a customer reference, a category, a timestamp, and a monetary amount.

A synthetic dataset is used deliberately rather than real customer records. Real transaction records contain personal data and are subject to data-protection obligations (UK GDPR); using a synthetic set preserves every structural property needed for this coursework while removing any privacy risk. As the brief notes, building a custom dataset does not earn additional marks, so the value here is purely in choosing a shape that supports rich, realistic operations.

Why this is a distributed application and not just a spreadsheet lookup (the "Beware the CRUD" point). The transactions ledger is written to concurrently by *four independent channels*. A customer taxing a car on the web, an agent registering a vehicle at a branch, and a post office clerk processing a change of keeper are all appending to and updating the *same* shared resource at the same

time. This is precisely the situation a single spreadsheet cannot handle safely: it needs a central authority that serialises writes, enforces consistency, computes aggregate figures (daily takings, per-customer spend, channel usage) on demand, and exposes a controlled interface to many simultaneous clients. The service therefore does far more than return single rows — it filters and sorts collections, and computes statistics such as date-range cost totals and top-spending customers.

1.2 Introducing the Dataset

The dataset is a single CSV file (`dvla_transactions_sample.csv`). Each **row** is one transaction and each **column** is a feature of that transaction. The sample provided with this submission contains **25 rows and 10 columns**; the production dataset is assumed to contain tens of thousands of rows of the same shape.

#	Feature (column)	Data type	Description	Example
1	`transaction_id`	String (identifier)	Unique primary key for the transaction.	`TXN-100013`
2	`customer_id`	String (foreign key)	Reference to the customer who made the transaction.	`CUST-4414`
3	`channel`	String (categorical / enum)	Service channel used. One of `web`, `phone`, `agent`, `post_office`.	`web`
4	`transaction_type`	String (categorical / enum)	Nature of the transaction. One of `vehicle_tax`, `sorn`, `change_of_keeper`, `first_registration`, `personalised_plate`, `driver_licence_renewal`.	`vehicle_tax`
5	`vehicle_type`	String (categorical / enum)	Class of vehicle. One of `car`, `motorcycle`, `hgv`, `bus`, `van`.	`motorcycle`
6	`vehicle_reg`	String	Vehicle registration mark (synthetic).	`UD73 BQO`
7	`transaction_date`	String (ISO 8601 datetime, UTC)	Timestamp the transaction was recorded.	`2026-02-28T15:26:00Z`
8	`amount`	Decimal (2 d.p.)	Fee paid, in minor-unit-safe decimal. `0.00` for free services such as SORN.	`531.95`
9	`currency`	String (ISO 4217)	Currency of `amount`. Always `GBP` in this dataset.	`GBP`
10	`status`	String (categorical / enum)	Processing state. One of `completed`, `pending`, `refunded`, `failed`.	`pending`

The dataset is **relational in spirit but flat in storage**: `customer_id` links many transactions to one customer, which is what makes the customer-scoped and aggregate endpoints in Task 2 meaningful.

1.3 Data Preprocessing

Because the source is a CSV file, preprocessing is required before the service can respond to client requests reliably. The CSV is loaded once at start-up into an in-memory indexed structure (and, in a production system, a database), with the following cleaning and restructuring steps:

- **Type coercion.** CSV stores everything as text. `amount` must be parsed to a fixed-precision decimal (never a binary float, to avoid rounding errors on monetary sums), and `transaction_date` must be parsed to a proper UTC datetime so that date-range queries and sorting work correctly.
- **Validation and normalisation of categoricals.** `channel`, `transaction_type`, `vehicle_type` and `status` are normalised to a fixed lower-case vocabulary so filters match consistently; `vehicle_reg` is upper-cased and whitespace-trimmed.
- **Integrity checks.** Rows with a missing or duplicate `transaction_id`, an unparseable date, or a negative `amount` are rejected or quarantined, because they would corrupt aggregate results.
- **Indexing for query performance.** To support the filtering, date-range, and top-N operations efficiently, the data is indexed by `transaction_id` (primary lookup) and by `customer_id` (customer-scoped queries), and kept sorted by `transaction_date` to make range scans cheap.
- **Referential consistency.** `customer_id` values are checked so that customer-scoped endpoints can distinguish "customer exists but has no transactions in range" (empty result, `200`) from "customer does not exist" (`404`).

This preprocessing is modest — a one-off parse and index at start-up — which satisfies the brief's consideration that cleaning should not be an excessive amount of work.

Task 2: Supported Operations and Endpoints

2.1 Defined Operations

The service supports **ten operations**. The brief requires at least seven, and at least one operation mapped to each of GET, POST, PUT and DELETE; this design satisfies both. Each operation is described below in natural language with its HTTP method and the reason that method is appropriate (drawing on the module's treatment of **safety** and **idempotency**: GET is safe and idempotent, PUT and DELETE are idempotent, POST is neither).

#	Operation (natural language)	HTTP method	Why this method
1	**List / filter transactions** — retrieve a collection of transactions, optionally filtered by channel, type, vehicle type, status, and date range, with sorting and pagination. *(complex return: multiple rows by selection criteria)*	`GET`	Safe, read-only, idempotent, cacheable.
2	**Get a single transaction** — retrieve one transaction by its id.	`GET`	Safe read of an individual resource.
3	**Create a transaction** — record a new transaction; the server assigns the `transaction_id`.	`POST`	Non-idempotent creation where the server mints the new URI (per the PUT-vs-POST guidance: "generally use POST").
4	**Replace a transaction** — fully update an existing transaction identified by its id.	`PUT`	Idempotent full replacement at a client-known URI.

5	**Update transaction status** — change only the processing status of a transaction (e.g. mark `refunded`).	`PUT`	Idempotent update of a single sub-resource (`/status`); re-sending the same status yields the same state.
6	**Void / delete a transaction** — remove a transaction from the ledger.	`DELETE`	Idempotent deletion of an identified resource.
7	**Customer cost total over a date range** — compute the sum of `amount` for one customer between two dates. *(complex return: sum of value between dates, by customer)*	`GET`	Safe, computed read; idempotent and cacheable.
8	**Top-N customers by spend** — return the top *N* customers ranked by total spend over a period. *(complex return: top-5 aggregation)*	`GET`	Safe aggregate read.
9	**Channel usage breakdown** — counts and totals grouped by channel over a period.	`GET`	Safe aggregate read.
10	**List a customer's transactions** — all transactions belonging to one customer (nested resource).	`GET`	Safe read of a sub-collection.

Method coverage: **GET** (1, 2, 7, 8, 9, 10), **POST** (3), **PUT** (4, 5), **DELETE** (6).

2.2 Endpoints

All endpoints share the root `http://localhost:8181/2356784/`. URIs name **resources (nouns)**, not actions, and follow a logical hierarchy (`customers/{id}/transactions`, mirroring the `customers/{id}/orders` pattern from the lectures). Optional filters, sorting and pagination are passed as **query parameters**; resource identity is passed as **path parameters** — exactly the convention recommended in the Parameter Passing lecture.

#	Operation	Method	Endpoint (relative to root)
1	List / filter transactions	`GET`	`/transactions?channel=web&type=vehicle_tax&status=pending&from=2026-01-01&to=202amount&page=1&size=20`
2	Get a single transaction	`GET`	`/transactions/{transactionId}`
3	Create a transaction	`POST`	`/transactions`
4	Replace a transaction	`PUT`	`/transactions/{transactionId}`
5	Update transaction status	`PUT`	`/transactions/{transactionId}/status`
6	Void / delete a transaction	`DELETE`	`/transactions/{transactionId}`
7	Customer cost total over range	`GET`	`/customers/{customerId}/transactions/cost?from=2026-01-01&to=2026-`

8	Top-N customers by spend	`GET`	`/stats/top-customers?metric=spend&limit=5&from=2026-01-01&to=2026-`
9	Channel usage breakdown	`GET`	`/stats/channels?from=2026-01-01&to=2026-06-30`
10	List a customer's transactions	`GET`	`/customers/{customerId}/transactions?status=completed`

Note that operations 2, 4 and 6 **reuse the same URI** (`/transactions/{transactionId}`) with different methods — the uniform-interface principle at work: one resource, many verbs. As the brief observes, this is why ten operations do not require ten endpoints.

Task 3: Request and Response Representations

The service uses **JSON** for all request and response bodies. JSON is chosen over XML because, as noted in the module, it is the informal convention for REST, is lighter and easier for clients to parse, maps directly onto the objects a Spring Boot service would (de)serialise via Jackson, and needs none of XML's namespace machinery. Clients indicate the format they send with the `Content-Type` header and the format they can accept with the `Accept` header.

For every operation below, the expected request header(s) and an example are given, followed by an example response. Collection responses are wrapped in an envelope carrying pagination metadata so the client can page through large result sets without guessing.

Operation 1 — List / filter transactions (`GET /transactions`)

No request body is needed; all input is in the URI. The client still sends an `Accept` header.

Request

```
GET /2356784/transactions?
channel=web&type=vehicle_tax&from=2026-02-01&to=2026-03-31&sort=-
amount&page=1&size=20 HTTP/1.1
Host: localhost:8181
Accept: application/json
```

Response `200 OK`

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "page": 1,
  "size": 20,
  "totalItems": 1,
  "totalPages": 1,
  "items": [
```

```
    {
      "transactionId": "TXN-100013",
      "customerId": "CUST-4414",
      "channel": "web",
      "transactionType": "vehicle_tax",
      "vehicleType": "motorcycle",
      "vehicleReg": "UD73 BQO",
      "transactionDate": "2026-02-28T15:26:00Z",
      "amount": 531.95,
      "currency": "GBP",
      "status": "pending"
    }
  ]
}
```

Justification: query parameters are ideal for optional filters, sorting and pagination; the envelope gives the client everything needed to render and navigate results (self-descriptive message).

Operation 2 — Get a single transaction (`GET /transactions/{transactionId}`)

Request

```
GET /2356784/transactions/TXN-100020 HTTP/1.1
Host: localhost:8181
Accept: application/json
```

Response `200 OK`

```
{
  "transactionId": "TXN-100020",
  "customerId": "CUST-4407",
  "channel": "web",
  "transactionType": "personalised_plate",
  "vehicleType": "motorcycle",
  "vehicleReg": "GH52 HOA",
  "transactionDate": "2026-03-25T11:54:00Z",
  "amount": 709.78,
  "currency": "GBP",
  "status": "completed"
}
```

Operation 3 — Create a transaction (`POST /transactions`)

The client sends a JSON body; it must **not** send `transactionId` because the server assigns it.

Request

```
POST /2356784/transactions HTTP/1.1
Host: localhost:8181
Content-Type: application/json
Accept: application/json
```

```
{
  "customerId": "CUST-4414",
  "channel": "web",
  "transactionType": "vehicle_tax",
  "vehicleType": "car",
  "vehicleReg": "AB12 CDE",
  "transactionDate": "2026-06-01T09:15:00Z",
  "amount": 180.00,
  "currency": "GBP",
  "status": "pending"
}
```

Response `201 Created`

```
HTTP/1.1 201 Created
Location: /2356784/transactions/TXN-100025
Content-Type: application/json
```

```
{
  "transactionId": "TXN-100025",
  "customerId": "CUST-4414",
  "channel": "web",
  "transactionType": "vehicle_tax",
  "vehicleType": "car",
  "vehicleReg": "AB12 CDE",
  "transactionDate": "2026-06-01T09:15:00Z",
  "amount": 180.00,
  "currency": "GBP",
  "status": "pending"
}
```

Justification: a request body is appropriate here because the client is sending a structured object; the `Location` header tells the client the URI of the newly minted resource.

Operation 4 — Replace a transaction (`PUT /transactions/{transactionId}`)

Request

```
PUT /2356784/transactions/TXN-100025 HTTP/1.1
Host: localhost:8181
Content-Type: application/json
Accept: application/json
```

```
{
  "customerId": "CUST-4414",
  "channel": "web",
  "transactionType": "vehicle_tax",
  "vehicleType": "car",
  "vehicleReg": "AB12 CDE",
  "transactionDate": "2026-06-01T09:15:00Z",
  "amount": 190.00,
  "currency": "GBP",
```

```
  "status": "completed"
}
```

Response `200 OK` — returns the full updated resource (same shape as above, with `amount` `190.00` and `status` `completed`).

Operation 5 — Update transaction status (`PUT /transactions/{transactionId}/status`)

A minimal body is used because only one field changes.

Request

```
PUT /2356784/transactions/TXN-100013/status HTTP/1.1
Host: localhost:8181
Content-Type: application/json
Accept: application/json
```

```
{ "status": "refunded" }
```

Response `200 OK`

```
{ "transactionId": "TXN-100013", "status": "refunded" }
```

Justification: exposing status as a sub-resource keeps the common "mark refunded" workflow simple and idempotent without requiring the client to resend the whole transaction.

Operation 6 — Void / delete a transaction (`DELETE /transactions/{transactionId}`)

No request body.

Request

```
DELETE /2356784/transactions/TXN-100025 HTTP/1.1
Host: localhost:8181
Accept: application/json
```

Response `204 No Content` (empty body).

Operation 7 — Customer cost total over a date range (`GET /customers/{id}/transactions/cost`)

Request

```
GET /2356784/customers/CUST-4414/transactions/cost?from=2026-01-01&to=2026-06-30
HTTP/1.1
Host: localhost:8181
Accept: application/json
```

Response `200 OK`

```
{
  "customerId": "CUST-4414",
  "from": "2026-01-01",
  "to": "2026-06-30",
  "currency": "GBP",
  "transactionCount": 5,
  "totalAmount": 600.95
}
```

Justification: returning a small computed object rather than the raw rows is far easier for the client to use — the client wanted a total, so the service does the aggregation server-side.

Operation 8 — Top-N customers by spend (`GET /stats/top-customers`)

Request

```
GET /2356784/stats/top-customers?metric=spend&limit=5&from=2026-01-01&to=2026-06-30
HTTP/1.1
Host: localhost:8181
Accept: application/json
```

Response `200 OK` (truncated to the top 5)

```
{
  "metric": "spend",
  "from": "2026-01-01",
  "to": "2026-06-30",
  "currency": "GBP",
  "results": [
    { "rank": 1, "customerId": "CUST-4407", "totalAmount": 955.06 },
    { "rank": 2, "customerId": "CUST-4413", "totalAmount": 608.66 },
    { "rank": 3, "customerId": "CUST-4414", "totalAmount": 600.95 },
    { "rank": 4, "customerId": "CUST-4401", "totalAmount": 381.37 },
    { "rank": 5, "customerId": "CUST-4409", "totalAmount": 110.00 }
  ]
}
```

Operation 9 — Channel usage breakdown (`GET /stats/channels`)

Response `200 OK`

```
{
  "from": "2026-01-01",
  "to": "2026-06-30",
  "currency": "GBP",
  "channels": [
    { "channel": "web",         "transactionCount": 5, "totalAmount": 1310.73 },
    { "channel": "phone",       "transactionCount": 7, "totalAmount": 734.66  },
    { "channel": "agent",       "transactionCount": 7, "totalAmount": 520.28  },
    { "channel": "post_office", "transactionCount": 6, "totalAmount": 462.37  }
  ]
}
```

Operation 10 — List a customer's transactions (`GET /customers/{id}/transactions`)

Response `200 OK` — same paginated envelope as Operation 1, filtered to the given customer.

Task 4: Error Handling and Status Codes

4.1 Common Errors and 4.2 Status Codes (per operation)

The service makes full use of HTTP status codes to describe outcomes. All error responses share a consistent JSON body so clients can handle them uniformly:

```
{ "timestamp": "2026-06-01T09:15:00Z", "status": 404, "error": "Not Found",
  "message": "No transaction with id TXN-999999", "path": "/2356784/transactions/
TXN-999999" }
```

Op	Method & endpoint	Likely errors (client / server)	Status codes used
1	`GET /transactions`	Invalid filter value (e.g. `channel=fax`), malformed date, `from` after `to`, bad pagination. Server fault during read.	`200 OK`; `400 Bad Request`; `500 Internal Server Error`.
2	`GET /transactions/{id}`	Transaction id does not exist; malformed id.	`200 OK`; `404 Not Found`; `400 Bad Request`.
3	`POST /transactions`	Missing required field, invalid enum, negative/invalid `amount`, bad date, wrong `Content-Type`, client wrongly sends `transactionId`. Duplicate submission.	`201 Created`; `400 Bad Request`; `409 Conflict`; `415 Unsupported Media Type`; `500`.
4	`PUT /transactions/{id}`	Id not found; body fails validation; body `transactionId` mismatches path.	`200 OK`; `400 Bad Request`; `404 Not Found`; `409 Conflict`.
5	`PUT /transactions/{id}/status`	Id not found; invalid status value; illegal transition (e.g. `failed` → `completed`).	`200 OK`; `400 Bad Request`; `404 Not Found`; `409 Conflict`.
6	`DELETE /transactions/{id}`	Id not found; already deleted.	`204 No Content`; `404 Not Found`.
7	`GET /customers/{id}/transactions/cost`	Customer not found; invalid/missing date range. (No matching rows is **not** an error → total `0.00`.)	`200 OK`; `400 Bad Request`; `404 Not Found`.
8	`GET /stats/top-customers`	Invalid `metric`, non-positive `limit`, bad dates.	`200 OK`; `400 Bad Request`.
9	`GET /stats/channels`	Invalid/missing date range.	`200 OK`; `400 Bad Request`; `500`.
10	`GET /customers/{id}/transactions`	Customer not found; invalid filter.	`200 OK`; `404 Not Found`; `400 Bad Request`.

Why each code is used:

- **200 OK** — a successful read or update that returns a representation.
- **201 Created** — a new transaction was created; paired with a `Location` header.

- **204 No Content** — a successful delete with nothing to return.
- **400 Bad Request** — the client sent syntactically or semantically invalid input (bad enum, bad date, `from` after `to`): the request itself is at fault.
- **404 Not Found** — the identified resource (transaction or customer) does not exist.
- **409 Conflict** — the request conflicts with current state: a duplicate create, a path/body id mismatch, or an illegal status transition.
- **415 Unsupported Media Type** — the client sent a body in a `Content-Type` the service does not accept (not `application/json`).
- **500 Internal Server Error** — an unexpected server-side fault (e.g. failure while reading the data store); the client did nothing wrong and may retry.

A distinction is drawn between a **client error (4xx)** — the caller must change the request — and a **server error (5xx)** — the caller may safely retry later. Importantly, an empty result set (a filter or date range that matches no rows) is a **successful `200`** with an empty collection or a zero total, **not** a `404`; `404` is reserved for a missing *resource*, not an empty *result*.

Task 5: Concurrency and Multithreading

The service is designed to handle many simultaneous clients, so the main thread accepts connections and dispatches each request to a worker thread (the standard Spring Boot / servlet model). The key design question is, for each operation, whether it can run concurrently and where shared mutable state creates **race conditions** on the one resource every channel contends over: the transactions ledger.

Read operations (1, 2, 7, 8, 9, 10)

All GET operations are **safe** (no side effects) and can be freely multithreaded; many reads can proceed in parallel. The only hazard is a **read–write race**: a reader computing an aggregate (e.g. Operation 7's date-range total, or Operation 8's top-N) while a writer is simultaneously creating, updating or deleting a transaction could observe a *torn* or inconsistent view — for example counting a transaction's old amount after part of an update has been applied, producing an incorrect total.

Mitigation: use a **read–write lock** on the ledger (many concurrent readers, exclusive writer) or a concurrent data structure that provides a consistent snapshot for the duration of an aggregate scan. Because aggregates (7, 8, 9) read many rows, they must see a single consistent snapshot rather than reading rows one at a time while writes interleave. Monetary sums must use fixed-precision decimals so that concurrent partial computations never introduce floating-point drift.

Write operations (3 create, 4/5 update, 6 delete)

Writes mutate shared state and are the main source of races:

- **Operation 3 (create):** two channels creating transactions at the same time could be assigned the **same `transaction_id`** if id generation is not atomic (a classic lost-update / duplicate-key race). *Mitigation:* generate ids with an atomic counter or UUIDs, and make the insert atomic. To guard against a client's double submission (e.g. a retried web request), accept an **idempotency key** so a repeated create returns the original resource instead of creating a duplicate.
- **Operations 4 and 5 (update / status change):** two agents updating the *same* transaction concurrently create a **lost-update race** — the second write silently overwrites the first; and an

illegal-transition race where two threads both read `pending` and each move it to a different terminal state. *Mitigation:* take a per-resource (row-level) exclusive lock for the read-modify-write, and/or use **optimistic concurrency** with a version field or `ETag` / `If-Match` header so a stale update is rejected with `409 Conflict` rather than clobbering newer data. Because PUT is idempotent, re-sending the *same* update is safe.

- **Operation 6 (delete):** a **delete–update race** (one thread deletes while another updates the same transaction) or a double-delete. *Mitigation:* the same per-resource lock serialises delete against update; a delete of an already-removed transaction simply returns `404/204` and is idempotent.

Overall strategy

Concurrency is maximised by locking at the **finest safe granularity**: shared reads use a shared lock, and writes lock only the specific transaction being modified (per-row locking) rather than the whole ledger, so unrelated channels do not block one another. Aggregate reads take a consistent snapshot. This keeps the common case — many independent reads and writes to *different* transactions — fully parallel, while guaranteeing data integrity for the rare case of true contention on the *same* transaction. The design focuses on data integrity and race prevention rather than low-level thread-scheduling detail, as the brief requests.

Summary

This design presents a DVLA-themed vehicle-transactions web service with ten RESTful operations spanning all four HTTP verbs, noun-based URIs rooted at `http://localhost:8181/2356784/`, JSON request/response representations with explicit headers, a complete per-operation error and status-code scheme, and a concurrency analysis grounded in the shared-ledger contention that justifies the service as a genuine distributed application. The accompanying `dvla_transactions_sample.csv` provides the concrete data behind every example above.